

Listener update-loop timeout storm and unplanned restart

Incident report — worker listener (Telegram MTProto) · TScopier production on Railway

Severity	High — fleet-wide listener instability	Status	Fixed — code complete on a local branch; not yet committed or deployed
Detected	2026-09-29 20:57 UTC (log review)	Reported	2026-09-29; report written 2026-09-30
Affected users	43 sessions in the storm window; 1 session permanently disconnected (b8cc4467-...)	Component	Worker listener — Telegram MTProto client
Environment	Railway production (Listener / Trade / Backtest)	Root cause	Telegram connections were disconnected but never destroyed, leaving background ping loops running forever; the ping-timeout handler was skipped exactly when it was needed

1. Executive summary (plain English): The copier keeps a permanent connection to Telegram for every user, and behind each connection a small background routine pings Telegram to keep the line alive. When a ping fails, Telegram's own library prints an error — and because we had only disconnected the connection instead of tearing it down, that ping routine kept running against a dead line, forever, for every connection we had ever used. In production this produced hundreds of error stacks per minute (about 10–15% of all log output) for at least four days.

On 29 September at 20:57 UTC a large group of connections failed at the same moment: 43 sessions tried to reconnect within a single second, the log service hit its rate limit and started dropping messages, and the worker container restarted about four minutes later (21:01–21:04 UTC). It was not a deployment — the last deployment had been 10.6 hours earlier.

For users this meant hours of growing staleness before the storm (up to roughly two hours), a few minutes of downtime around the restart, and one Telegram session that could not be repaired at all — that user must re-link Telegram. After the restart the listener was dispatching signals again at 22:03–22:09 UTC.

The fix tears down dead connections properly, reacts to every ping timeout instead of ignoring them, spaces out reconnects so a mass failure cannot stampede, and stops retrying forever when Telegram has revoked a login key — it tells the user to re-link instead.

2. Issue encountered

From the Railway production logs (Listener service):

- Chronic noise:** Error: TIMEOUT from the library's update loop (telegram/client/updates.js) appears in every sampled window for at least four days — 263 lines / 2 min on 09-25 12:00, 1 011 on 09-28 00:00, 797 at 09-29 20:55, **935 / 10 min** at 22:05. That is roughly **10–15% of all log lines**, and the worker produced **zero** of its own "requesting reconnect" lines alongside them.
- The storm (20:57:45–20:58:14 UTC):**
 - 43 unique sessions logged _updateLoop TIMEOUT ... requesting reconnect in the second 20:57:45 (291 lines), then 43 connect_start events at 20:57:49 with source=update_loop_timeout , connection generations 2–10.
 - 138 raw Error: TIMEOUT stacks burst at 20:58:00.016–.207.
 - Railway's 500 logs/second cap was hit six times; 66 + 2 192 messages were dropped, so the export is ragged and the event looks worse than the data proves.
- Staleness before the storm:** at 20:57:49 the largest lastEventAgeMs values were 7 579 s (2.1 h), 2 601 s, 2 202 s, 517 s, 353 s, 321 s — sessions had been going quiet for hours.
- Restart:** at 21:01–21:04 UTC, 46 sessions logged connect_start with source=initial and uptimeMs reset to ~60 160 ms. The last deployment was **2026-09-29T10:19:08Z** (10.6 h earlier), so this was not deploy-triggered. The exit reason is unavailable — the Railway token cannot read metrics or deploymentLogs .
- One permanently dead session:** user b8cc4467-8f1a-4a69-ba5b-7cc750c529d2 logged 401 AUTH_KEY_UNREGISTERED (415 lines / 10 min) on four channels; the watchdog reconnected it every ~30 s forever.
- Not the usual suspects:** zero flood wait , zero AUTH_KEY_DUPLICATED — neither the July flood-wait storm nor duplicate sessions.
- After the restart:** dispatch signal at 22:03, 22:06, 22:06, 22:09 with 46 sessions heartbeating connected=true .

3. Affected user(s)

Scope	Who	What they experienced	DB state
Storm window	43 sessions (13 distinct users at 20:57:45)	Connections dropped and reconnected inside one second; copying paused while they recovered	Recovered after the restart
Fleet-wide	All sessions on the Listener service	Hours of growing staleness (up to 2.1 h), then ~3–5 min of downtime around 21:01–21:04 UTC	Recovered; 46 sessions connected=true by 22:09
Permanent	b8cc4467-8f1a-4a69-ba5b-7cc750c529d2	No copying until the user re-links Telegram	401 AUTH_KEY_UNREGISTERED , 415 lines / 10 min

Honest limit: we could not prove that any specific user missed a copied trade. Signal dispatch was observed after the restart, but log volume in the window was too low (and partly dropped by the log rate cap) to demonstrate per-session delivery.

4. Root cause

- Orphaned background ping loops.** The library's update loop runs while (!client.destroyed) and only exits when destroy() has been called; disconnect() does not set that flag and does not stop the loop (verified in worker/node_modules/telegram/client/updates.js and telegramBaseClient.js). The worker only ever called disconnect() — in stop() , in forceReconnect() and in the recovery paths — and **never** destroy() . The library's own fallback (client.sender.reconnect()) was patched to a no-op by buildClient() whenever autoReconnect: false , so the loop could not recover either. Every client that had ever been connected kept pinging a dead socket for the life of the process: ~2 stacks/minute each (backtest runs were hit hardest — 560 stacks / 10 min, listener 375 / 10 min).
- The timeout handler was skipped in the exact situation it exists for.** The handler was guarded with msg.includes('TIMEOUT') && this.isConnected . When a connection stalls, isConnected is already false , so no reconnect was requested and nothing was logged by us — only the library's raw stack. That guard was introduced by commit 4a0febe06 (2026-07-28), replacing the correct guard added on 2026-07-24 (docs/PROJECT_MEMORY.md:2069), and the await on the reconnect was dropped at the same time, so recovery raced the still-running ping loop.
- Simultaneous failure → stampede.** When many sessions lost their connection in the same second, all 43 requested a reconnect at once with a fixed delay and no jitter: 291 log lines in one second, saturating the 500/s log pipeline.
- A revoked key is unrecoverable.** 401 AUTH_KEY_UNREGISTERED cannot be fixed by reconnecting, so the watchdog retried every ~30 s forever without telling the user anything actionable.
- What restarted the container at ~21:01Z is still unknown** — no deployment was in progress and the token cannot read exit metrics.

5. The fix

- React to every update-loop timeout.** The handler now recognises the ping timeout by message alone (no isConnected guard) and **awaits** the full reconnect cycle, so the ping loop and the recovery no longer race.
- Tear connections down properly.** destroy() is now called when a listener stops and when a recovery replaces its client (with a bounded disconnect() fallback), and ephemeral backtest clients are destroyed in a finally . A client that will be reused is still reconnected rather than destroyed, because the library starts its update loop only once per client.
- Space out reconnects.** The reconnect delay carries a random jitter (TELEGRAM_RECONNECT_JITTER_MAX_MS , default 2000 ms, clamped 0–30 000), so a mass failure spreads across a second or

- two instead of landing in one tick.
4. **Stop retrying a revoked key.** A `sessionRevoked` flag is set the moment Telegram reports the key as unregistered — from the watchdog, a reconnect cycle, or a duplicated-key recovery. It short-circuits every reconnect path, destroys the client, keeps the lease without the hard-reset cycle that made the re-link banner flicker, and writes `telegram_account_status = reconnect_required` with `recovery_exhausted = true` (the existing re-link banner). Callers get a plain error: "Telegram connection expired. Reconnect Telegram to resume copying."
 5. **Stop the log flood.** The raw `Error: TIMEOUT` stacks are aggregated to one line per minute (both `console.log` and `console.error` are patched at startup, before the Sentry pipeline initialises), and `/Error: TIMEOUT\b/` was added to Sentry's log-noise filter. Our own actionable line `[userListener] _updateLoop TIMEOUT for <user> - requesting reconnect` does not match the pattern and is preserved.
 6. **Config safety.** A shared `envNumber()` helper now parses the clamped numeric settings (reconnect cooldown, jitter, connect timeout, auth-duplicate delays, healing ticks). A malformed value used to produce `NaN`, which silently disables the arithmetic it sits in — a typo in `.env` could have removed a cooldown or a timeout entirely.

6. Files changed

File	Change
worker/src/userListener.ts	Timeout handler (no <code>isConnected</code> guard, awaited reconnect); <code>destroy()</code> on stop and on replaced clients; <code>sessionRevoked</code> flag + <code>noteSessionRevoked()</code> ; sticky <code>sessionInvalid</code> health write with <code>allowWithoutLease</code> ; gates on <code>requestReconnect / requestReconnectIfDisconnected / runWatchdog / ensureTelegramConnected</code> ; <code>forceReconnect</code> abandons a cycle that learned the key was revoked mid-flight; duplicated-key recovery stops and reports the re-link error; cooldown/jitter helpers
worker/src/sessionManager.ts	Lease renew keeps a revoked listener parked (no hard reset, no per-tick log) and skips the "listener quiet" warning; <code>disconnectListener()</code> preserves <code>reconnect_required</code> and <code>recovery_exhausted</code> instead of hard-coding <code>linked</code> ; bounded <code>destroy()</code> when adopting a client fails; numeric env parsers routed through <code>envNumber()</code>
worker/src/backtestSync.ts	Ephemeral Telegram client destroyed in <code>finally</code> (bounded fallback)
worker/src/telegramClient.ts	<code>isSessionInvalid()</code> — recognises both shapes of a revoked key
worker/src/gramjsLogSuppress.ts	Patches <code>console.error</code> as well as <code>console.log</code> ; <code>isGramjsUpdateLoopTimeout()</code> ; 60-second aggregation line routed through the live logger
worker/src/observability/sentry.ts	<code>/Error: TIMEOUT\b/</code> added to <code>DEFAULT_LOG_NOISE_PATTERNS</code>
worker/src/authKeyDuplicatedRecovery.ts	Reconnect delay / max attempts / deferred retry parsed with <code>envNumber()</code>
worker/src/envNumber.ts	New — shared clamped numeric env parser (falls back on <code>NaN</code>)
worker/src/userListener.updateLoopRecovery.test.ts	New — 16 regression tests
worker/src/sessionManager.revokedHealth.test.ts	New — 3 tests pinning the health writes on stop and the parked-renew behaviour
worker/src/sessionManager.shutdown.test.ts	Fast-recovery fixture pins the jitter env so the recovery test stays deterministic
worker/src/index.ts	Comment updated for the <code>console.error</code> patch
worker/.env.example	<code>TELEGRAM_RECONNECT_JITTER_MAX_MS</code> documented

7. Verification

- `npm --prefix worker run build` (TypeScript) — clean.
- `npx eslint` over every changed file — clean.
- Targeted suites: `userListener.updateLoopRecovery` + `authKeyDuplicatedRecovery` + `copierHealth` + `observability/sentry` = **92/92 pass**; `sessionManager.shutdown` + `sessionManager.realtime` = **20/20 pass**.
- Full worker suite after the final edits: **1980 tests, 1980 pass, 0 failures** (1977 existing + the 3 new health-write tests). The previous revision ran 1976/1977; its single failure was the jitter test's own upper bound, since replaced by a deterministic assertion.
- Two review passes were run by the `code-review` subagent. The first found 1 HIGH and 5 MEDIUM; the second found 2 HIGH (one already fixed, one the flaky test) plus MEDIUM/LOW items — all addressed and re-verified above.

8. Deployment status

Branch: `fix/listener-update-loop-recovery` (from `origin/staging`), squashed into one commit and pushed to **origin/staging and upstream/staging** on 2026-09-30. Railway redeploys the staging Listener/Trade services from `staging`.

Not yet in production. `main` is untouched; promote only after the staging worker is validated in the Railway logs (no raw `Error: TIMEOUT` stacks, and `reconnect_required` rows behaving as described).

Local-only extras in the same working tree (not part of this incident): `scripts/railway-logs.py` and the `railway-logs` skill used to read the logs.

9. Follow-ups

1. **Log export limits:** the Railway token used for the investigation cannot read `metrics` or `deploymentLogs`, so the 21:01Z restart has no exit reason. A broader read-only token (or the Railway dashboard) is needed to close this.
2. **Separate defect found during the investigation:** the per-channel config heal path fails with `null` value in column `"copy_limit_state"` (32 occurrences / 10 min, 8 distinct channels), silently defeating "heal missing config from defaults". Not fixed here.
3. **Separate defect found during the investigation:** two broker accounts (`48b2cdfa-...`, `87383068-...`) are offline in `FxSocket` — 104 quote failures each per 10 min, 498 + 335 deferred ghost closes, and 322 `OrderModify` retries against one permanently rejected ticket with no backoff. Not fixed here.
4. **Alerting:** consider an alert on the aggregated update-loop timeout rate (now one line per minute) so a recurrence is visible before it becomes a storm.